

Překlad latinské Javy do instrukční sady PL/0

Jiří Třesohlavý, Jan Vašátko

21. ledna 2025

Popis práce

- Úprava jazyka Java do latinské podoby (Iava)
 - Přepsání klíčových slov (`for`, ...) do latiny (`pro`, ...)
- Vytvoření abstraktního syntaktického stromu AST
- Provedení sémantické analýzy (má program vůbec smysl?)
- Generování instrukcí z instrukční sady PL/0

Ukázka jazyka

```
1  class Trida {
2      boolean bool = verus || 1 >= 0;
3      finalis float a = 3;
4      char c = 3;
5      finalis int ahoj = 42;
6
7      vacuum metoda() {
8          //a += 1f;
9          int a = 32;
10         a++;
11     }
12
13     vacuum ahoj() {
14         boolean b = ahoj == 3;
15
16         si (b) {
17             int promenna = 3;
18             int promenna2 = promenna * promenna + 23;
19         }
20
21         redi;
22     }
23 }
```

Použité technologie

- Pro lexikální a syntaktickou analýzu – ANTLR
- ANTLR povolí přepsat klíč. slova z angličtiny do latiny
- maven natáhne si ANTLR závislosti a sestaví projekt
- Java nabízí širokou podporu optimalizovaných dat. struktur

Lexikální analýza

- Vytvoří proud tokenů, které jsou posléze použity pro synt. analýzu
- Byla potřeba přepsat klíčová slova Javy do latiny
- Práci nám usnadňuje technologie ANTLR
- Lexikální analýzu lze detailněji zkoumat v souboru `JavaLexer.g4`

JavaLexer.g4 – přepsání klíč. slov

```
1  ...
2  CASE      : 'casus';
3  CHAR      : 'char';
4  CLASS     : 'class';
5  CONTINUE  : 'persevera';
6  DEFAULT   : 'defaltam';
7  DO        : 'face';
8  FINAL     : 'finalis';
9  FLOAT     : 'float';
10 ...
```

Ukázka přepsání některých klíč. slov do latiny v souboru JavaLexer.g4

JavaLexer.g4 – definice čís. literálů

```
1  ...
2  DECIMAL_LITERAL : '0' | [1-9] (Digits? | '_' + Digits);
3  HEX_LITERAL     : '0' [xX] [0-9a-fA-F] ([0-9a-fA-F_]* [0-9a-fA-F])?;
4  OCT_LITERAL     : '0' '_'* [0-7] ([0-7_]* [0-7])?;
5  BINARY_LITERAL  : '0' [bB] [01] ([01_]* [01])?;
6
7  FLOAT_LITERAL:
8      (Digits '.' Digits? | '.' Digits) ExponentPart? [fF]?
9      | Digits (ExponentPart [fF]? | [fF])
10 ;
11
12 HEX_FLOAT_LITERAL: '0' [xX] (HexDigits '.'? | HexDigits? '.' HexDigits) [pP]
13     [+]? Digits [fF]?;
14 ...
```

Ukázka přepsání některých čís. literálů do latiny v souboru JavaLexer.g4

Syntaktická analýza

- Cíl: Vytvoření AST pomocí technologie ANTLR
- ANTLR již nabízí jistou formu AST – pro další kroky ale téměř nepoužitelné
- Pro další kroky byl vytvořen vlastní AST
- Vytvořili jsme vlastní struktury, které umožňují snadnější sémantickou analýzu a generování kódu
- ANTLR nabízí způsob (viz další slide), jak tento problém realizovat

Visitor

- Snadné navštěvování jednotlivých konstrukcí zdroj. kódu

```
1 public class ForLoopControlVisitor extends IJavaParserBaseVisitor<ForLoopControl>
2 {
3
4     @Override
5     public ForLoopControl visitForControl(IJavaParser.ForControlContext ctx) {
6         IJavaParser.ForInitContext forInitContext = ctx.forInit();
7
8         ForInit forInit = forInitContext == null
9             ? createEmptyForInit()
10             : extractForInit(ctx);
```

- Lze si jednoduše tímto způsobem vytvářet vlastní struktury, které budou součástí AST

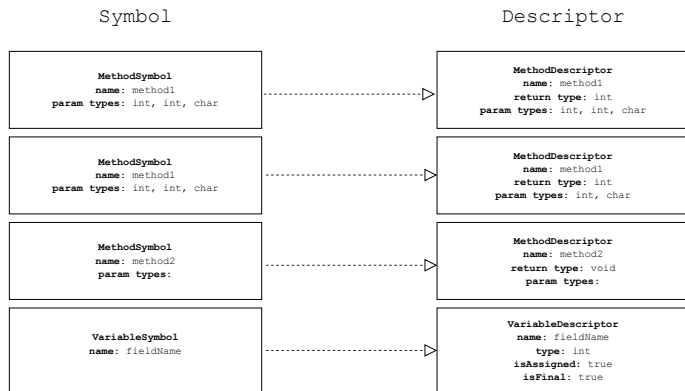
Sémantická analýza

- Kontrola, zda má program vůbec smysl
- Drželi jsme se regulí jazyka Java
- Byla potřeba implementace tabulky symbolů

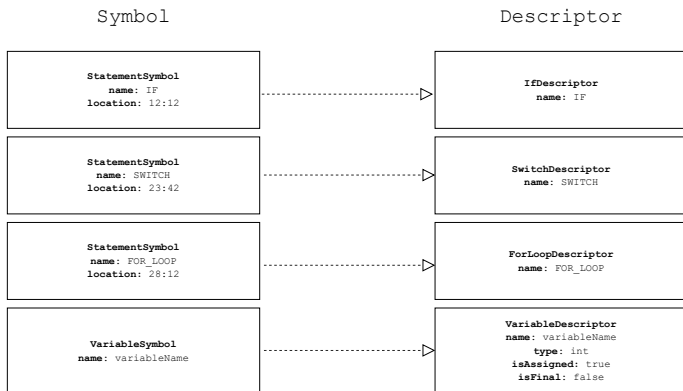
Tabulka symbolů

- Jedná se o n -ární strom scopů
- Každý scope si drží svůj parent scope
- Každý scope si drží n potomků – scopů
- Každý scope si drží svou tabulku (mapu) nadeklarovanych proměnných

Scope pro třídy



Scope pro bloky



Generování kódu

- Přeložení zdrojového kódu do instrukční sady PL/0
- Každý uzel AST odpovídá určité sadě instrukcí
- Generování kódu vyžaduje přesné řízení zásobníkových rámců
- Operace jsou prováděny na hodnotách uložených na vrcholu zásobníku
- Výsledné instrukce musí zajistit iterativní generování podle pravidel Javy

Zhodnocení

- Sémantická a syntaktická analýza se nám povedla dle pravidel Javy
- Povedlo se nám generovat kód pro cykly `for`, `while`, `do-while`
- Pro podmínky `if`, `if-else`, `switch-case-default`
- Pro metody – bezparametrické a parametrické
- základní indexování polí
- Pro zpracování jakýchkoli výrazů
 - `a++`, `++a`, `a + b - 2 * (a - 4)`, atd.

Konec

Děkujeme za pozornost